Task: Build a Vehicle Rental Management Backend

Objective

Build a small REST API for a vehicle rental company. Staff log in and manage the vehicle fleet; customer bookings are recorded as rentals. A vehicle can't be booked twice for overlapping dates. Provide a monthly report of rental activity per vehicle.

Requirements

1. Environment Setup

- Node.js + TypeScript project, OOP structure — services/classes, not everything crammed into route handlers
- Express as the web framework
- Knex as the query builder
- PostgreSQL (preferred) or MySQL — either way you'll need real SQL, not just Knex's chain syntax
- Joi or express-validator for input validation
- ESLint + Prettier configured
- .env for DB credentials, JWT secret, upload path, and port — gitignored, with a committed .env.example
- Multer (or similar) for local photo storage

2. Database Schema

`staff`

- `id` (PK, auto-increment)
- `email` (unique, required)
- `password_hash` (required)
- `name` (required)
- `created_at, updated_at`

`vehicles`

- `id` (PK, auto-increment)
- `name` (required)
- `plate_number` (unique, required)
- `category` (required)
- `daily_rate` (decimal, required)
- `photo_path` (optional)
- `deleted_at` (nullable)
- `created_at, updated_at`

`rentals`

- `id` (PK, auto-increment)
- `vehicle_id` (FK → vehicles.id, required)
- `customer_name` (required)
- `customer_phone` (required)
- `start_date` (date, required)
- `end_date` (date, required)
- `total_amount` (decimal, required)
- `status` (booked / ongoing / completed / cancelled — default booked)
- `created_at, updated_at`

Note: there's no column-level constraint that stops double-booking — two rentals only conflict if their date ranges actually overlap and both are active. That check belongs in your application code, on both create and update.

3. Authentication

- POST `/auth/login` — email + password → JWT
- JWT middleware protects every /vehicles, /rentals, and /reports route

4. Endpoints

`Auth`

- POST `/auth/login`

`Vehicles`

- GET `/vehicles` — pagination, filter by category, search by name
- GET `/vehicles/:id`
- POST `/vehicles` — multipart form-data with photo
- PUT `/vehicles/:id` — including photo replacement
- DELETE `/vehicles/:id` — soft delete

`Rentals`

- GET `/rentals` — filter by vehicle_id, status, and date range
- GET `/rentals/:id`
- POST `/rentals` — body: vehicle_id, customer_name, customer_phone, start_date, end_date
  - 409 if the vehicle already has an active rental overlapping these dates
  - total_amount is calculated server-side — daily_rate × number of days (same start/end date counts as 1 day)
- PUT `/rentals/:id` — date changes re-trigger the overlap check
- DELETE `/rentals/:id`

`Reports`

- GET `/reports/rentals?month=YYYY-MM` — optional &vehicle_id=
  - per vehicle — id, name, total_bookings, days_rented, revenue
  - only count days/revenue that fall inside the requested month — a rental running July 29–Aug 3 contributes 3 days to the August report, not 6
  - also return the vehicle with the highest revenue that month

5. TypeScript

- Type every request body, response, and handler return value
- Extend Express's Request type with the decoded JWT payload

6. Database Connection

- pg or mysql2 behind a Knex connection pool; pool size and credentials from env vars
- Migrations + seeds — seed at least one rental that spans a month boundary so the report is actually testable

Bonus (Optional)

- Wrap the availability check and insert in a transaction, so two people booking the same vehicle at the same moment can't both succeed
- Pagination/search on /rentals as well
- Basic rate limiting on /auth/login

Deliverables

- Public Git repository

- README with setup and run instructions
- .env.example
- Migration files — schema should build cleanly on an empty database

You should be ready to walk through your overlap and report queries and explain why they're correct — that's part of the review.